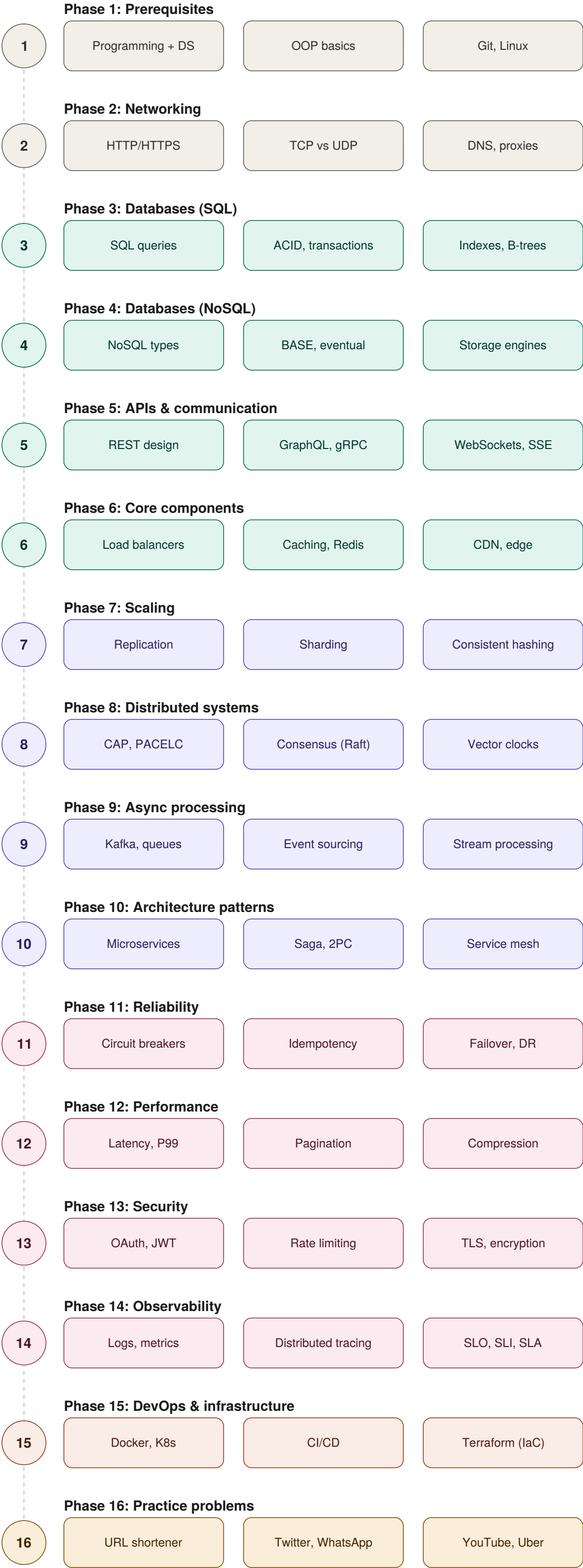


The Ultimate System Design Roadmap

A 16-phase guide for absolute beginners



Detailed Phase-by-Phase Breakdown

Every concept, key question, and free resource for the 16-phase system design roadmap. Refer to the diagram on page 1 for the visual overview.

Phase 1: Prerequisites (1 to 2 weeks)

What you learn: any one programming language deeply (Python, Java, Go, or JavaScript work), arrays, hash maps, linked lists, trees, graphs, basic recursion, time and space complexity (Big O), object-oriented programming (classes, inheritance, polymorphism, encapsulation), Git fundamentals (clone, branch, merge, rebase, conflicts), Linux command line (cd, ls, grep, awk, sed, ssh, chmod), basic shell scripting, IDE setup (VS Code, IntelliJ).

Key questions: When does $O(n^2)$ become a problem? Why is hash map lookup $O(1)$? When would you use composition over inheritance?

Free resource: freeCodeCamp's full Python or Java course on YouTube plus MIT 6.006 lecture videos.

Phase 2: Networking and the Internet (1 to 2 weeks)

What you learn: OSI 7-layer model, TCP/IP stack, TCP three-way handshake, TCP vs UDP tradeoffs, HTTP/1.1 vs HTTP/2 (multiplexing) vs HTTP/3 (QUIC), HTTPS and TLS handshake, DNS resolution flow (recursive vs iterative), A records, CNAME, MX records, IP addressing, subnetting basics, ports and sockets, WebSockets vs Server-Sent Events vs long polling vs short polling, CORS, forward proxies vs reverse proxies, network latency vs bandwidth vs throughput, packet loss, TTL.

Key questions: Why is DNS lookup so often the bottleneck? What happens between typing a URL and seeing the page? When would you choose UDP over TCP?

Free resource: Hussein Nasser's networking playlists plus High Performance Browser Networking (free online book by Ilya Grigorik).

Phase 3: Databases (SQL fundamentals) (2 weeks)

What you learn: relational model, SQL queries (SELECT, JOIN, GROUP BY, window functions, CTEs), normalization (1NF, 2NF, 3NF, BCNF), denormalization tradeoffs, ACID properties, transactions, isolation levels (Read Uncommitted, Read Committed, Repeatable Read, Serializable, Snapshot), dirty reads, phantom reads, indexes (B-tree, hash, bitmap, covering indexes, composite indexes), query optimization, EXPLAIN plans, connection pooling, prepared statements, foreign keys, the N+1 query problem.

Key questions: Why does adding an index make reads faster but writes slower? When does a JOIN become catastrophically slow? What is the difference between Repeatable Read and Serializable?

Free resource: Use The Index, Luke! (use-the-index-luke.com, completely free) plus Postgres documentation tutorial.

Phase 4: Databases (NoSQL and storage) (1 to 2 weeks)

What you learn: document stores (MongoDB), key-value stores (Redis, DynamoDB), wide-column stores (Cassandra, HBase), graph databases (Neo4j), time-series (InfluxDB, TimescaleDB), vector databases (Pinecone, Weaviate, pgvector), object storage (S3 model), block vs file vs object storage, BASE properties, eventual consistency, storage engines (LSM trees vs B-trees, write amplification, read amplification), tombstones, compaction, write-ahead log (WAL), MVCC.

Key questions: Why is Cassandra fast at writes but slow at reads? When would you pick MongoDB over Postgres? What is an LSM tree and why does it power so many modern databases?

Free resource: Database Internals book by Alex Petrov (free chapters available) plus Arpit Bhayani's database deep-dive videos.

Phase 5: APIs and communication (1 to 2 weeks)

What you learn: REST principles (statelessness, resource-based URLs, HATEOAS), HTTP methods (GET, POST, PUT, PATCH, DELETE), idempotent vs safe methods, status codes (2xx/3xx/4xx/5xx), API versioning strategies (URL, header, query param), GraphQL (queries, mutations, subscriptions, N+1 resolver problem, DataLoader), gRPC and Protocol Buffers, when to use REST vs GraphQL vs gRPC, WebSockets for bidirectional, Server-Sent Events for one-way streams, webhooks, API gateways (Kong, AWS API Gateway), API documentation (OpenAPI/Swagger), pagination patterns (offset, cursor, keyset).

Key questions: Why does Netflix use gRPC internally but REST externally? When does GraphQL hurt you instead of helping? How do webhooks differ from polling?

Free resource: Google's API Design Guide (free online) plus ByteByteGo's API videos.

Phase 6: Core building blocks (2 weeks)

What you learn: web servers (Nginx, Apache), application servers, load balancers (Layer 4 TCP vs Layer 7 HTTP), load balancing algorithms (round robin, weighted round robin, least connections, IP hash, least response time, consistent hashing for stateful), reverse proxies, caching (cache-aside, write-through, write-back, write-around, read-through), eviction policies (LRU, LFU, FIFO, TTL), Redis vs Memcached, distributed cache, local in-memory cache, HTTP caching headers (Cache-Control, ETag, Last-Modified), CDN (Cloudflare, Akamai, CloudFront, Fastly), edge computing, cache invalidation strategies, cache stampede, thundering herd.

Key questions: Why is cache invalidation famously one of the two hardest problems in computer science? When does a CDN actually slow you down? Why might you choose Memcached over Redis?

Free resource: System Design Primer GitHub (donnemartin/system-design-primer) plus Cloudflare's free engineering blog.

Phase 7: Scaling fundamentals (2 weeks)

What you learn: vertical scaling vs horizontal scaling tradeoffs, stateless vs stateful services, auto-scaling, master-slave replication, master-master replication, quorum-based replication, replication lag, read replicas, sharding strategies (range-based, hash-based, geo-based, directory-based), consistent hashing (the ring, virtual nodes), database federation, denormalization for reads, materialized views, CQRS pattern (Command Query Responsibility Segregation), partitioning, hot partitions, rebalancing.

Key questions: Why does Instagram shard by user ID but Discord shards by guild ID? What is the rebalancing cost when you add a node to a consistent hash ring? When does sharding hurt more than help?

Free resource: ByteByteGo's scaling playlist plus Vitess documentation (YouTube's sharding solution, free and well-documented).

Phase 8: Distributed systems theory (3 to 4 weeks)

What you learn: CAP theorem (Consistency, Availability, Partition tolerance, pick 2), PACELC extension (Else: Latency vs Consistency), consistency models (linearizable, sequential, causal, eventual, strong eventual), consensus algorithms (Paxos, Raft, ZAB), leader election, two-phase commit (2PC), three-phase commit (3PC), vector clocks, Lamport timestamps, gossip protocols, conflict resolution (last-write-wins, CRDTs), service discovery (Consul, etcd, Zookeeper), heartbeats, failure detection (phi accrual), split brain, Byzantine fault tolerance basics.

Key questions: Why can you not have all three of CAP simultaneously during a partition? How does Raft elect a leader? What is a CRDT and why does Figma love them?

Free resource: MIT 6.824 Distributed Systems lectures on YouTube (free and the gold standard) plus the Raft paper (raft.github.io).

Phase 9: Asynchronous processing (2 to 3 weeks)

What you learn: message queues (RabbitMQ, ActiveMQ, AWS SQS), event streaming (Kafka, Pulsar, AWS Kinesis), pub/sub patterns, producer/consumer model, partitions, offsets, consumer groups, dead letter queues, exactly-once vs at-least-once vs at-most-once delivery, backpressure handling, event-driven architecture, event sourcing, CQRS with event sourcing, stream processing (Kafka Streams, Apache Flink, Spark Streaming), batch processing (Spark, Hadoop MapReduce), Lambda architecture, Kappa architecture, workflow orchestration (Apache Airflow, Temporal, AWS Step Functions).

Key questions: Why is Kafka not really a queue? What does "exactly-once" actually mean in distributed systems? When would you use Temporal over Airflow?

Free resource: Confluent's free Kafka courses (developer.confluent.io) plus Tyler Akidau's stream processing series on the Apache Beam blog.

Phase 10: Architecture patterns (2 weeks)

What you learn: monolith (when it is the right call), microservices (when it is not), service-oriented architecture (SOA), serverless and Functions as a Service (Lambda, Cloud Run), modular monolith, strangler fig pattern (migrating monolith to microservices), Backend for Frontend (BFF), API gateway pattern, sidecar pattern, service mesh (Istio, Linkerd, Consul Connect), hexagonal architecture (ports and adapters), domain-driven design (DDD: bounded contexts, aggregates, ubiquitous language), saga pattern (choreography vs orchestration), outbox pattern, change data capture (Debezium).

Key questions: What is the actual cost of moving from monolith to microservices? Why did Amazon and Netflix go microservices but Stack Overflow stayed monolith? What problem does a service mesh solve?

Free resource: Sam Newman's Building Microservices (free chapters) plus Martin Fowler's microservices.io patterns catalog.

Phase 11: Reliability and fault tolerance (1 to 2 weeks)

What you learn: high availability (5 nines, 4 nines, what they actually mean), redundancy, active-passive vs active-active failover, disaster recovery (RPO: data loss tolerance, RTO: downtime tolerance), circuit breakers (Hystrix pattern, Resilience4j), bulkheads, retries with exponential backoff, jitter, idempotency keys, idempotent API design, timeouts, graceful degradation, chaos engineering (Chaos Monkey, Gremlin), multi-region deployments, geo-replication, health checks (liveness, readiness, startup), blue-green deployments, canary deployments, feature flags (LaunchDarkly, Unleash).

Key questions: Why does retrying without jitter make outages worse? What is the difference between graceful degradation and a circuit breaker? Why is idempotency the foundation of every reliable distributed system?

Free resource: Google SRE Book (free at sre.google/books) plus AWS Architecture Center disaster recovery whitepapers.

Phase 12: Performance and optimization (1 week)

What you learn: latency vs throughput tradeoffs, percentiles (P50/median, P95, P99, P99.9), tail latency, capacity planning, performance benchmarking (k6, JMeter, wrk), profiling (CPU, memory, flame graphs), connection pooling, compression (gzip vs brotli vs zstd), lazy loading, eager loading, pagination strategies (offset is bad at scale, use cursor/keyset), N+1 query problem, batching (Facebook DataLoader pattern), debouncing, throttling, hot spots, write amplification, read amplification.

Key questions: Why is P99 more important than average latency? What is tail latency and why does Jeff Dean care so much about it? When does pagination become catastrophic?

Free resource: Brendan Gregg's website on performance plus Designing Data-Intensive Applications Chapter 1 (often available free online).

Phase 13: Security at scale (1 to 2 weeks)

What you learn: authentication (passwords, MFA, biometric, magic links, passkeys), authorization (RBAC, ABAC, ACL), OAuth 2.0 flows (authorization code, implicit, client credentials, PKCE), OpenID Connect, JWT structure (header, payload, signature), session management, refresh tokens, HTTPS/TLS handshake, certificate pinning, encryption at rest, encryption in transit, hashing (bcrypt, scrypt, Argon2), API security (rate limiting algorithms: token bucket, leaky bucket, fixed window, sliding window log, sliding window counter), DDoS protection, WAF, SQL injection prevention, XSS, CSRF, secrets management (Vault, AWS Secrets Manager), zero trust architecture.

Key questions: Why is bcrypt better than SHA-256 for passwords? What is the actual difference between authentication and authorization? Why is OAuth so confusing and what problem does it solve?

Free resource: OWASP Top 10 (free and authoritative) plus Auth0's free blog posts on identity.

Phase 14: Observability (1 week)

What you learn: structured logging (JSON logs, log levels, log aggregation), metrics (counters, gauges, histograms, summaries), Prometheus model, distributed tracing (OpenTelemetry, Jaeger, Zipkin, span context propagation), APM tools (Datadog, New Relic, Honeycomb), the three pillars of observability, Service Level Indicators (SLI), Service Level Objectives (SLO), Service Level Agreements (SLA), error budgets, alerting (PagerDuty, Opsgenie), runbooks, on-call rotation, incident response (incident commander role, postmortems, blameless culture), the USE method, the RED method.

Key questions: Why does log aggregation matter once you have more than two servers? What is the difference between metrics, logs, and traces, and when do you need each? Why is a 99.9% SLO not 99.9% uptime?

Free resource: OpenTelemetry official docs plus Honeycomb's free O'Reilly book on Observability Engineering.

Phase 15: DevOps and infrastructure (2 weeks)

What you learn: containerization (Docker images, layers, multi-stage builds, Dockerfile best practices), container orchestration (Kubernetes: pods, deployments, services, ingress, ConfigMaps, secrets), CI/CD pipelines (GitHub Actions, GitLab CI, Jenkins, CircleCI), GitOps (ArgoCD, Flux), infrastructure as code (Terraform, Pulumi, AWS CDK), configuration management (Ansible, Chef), cloud providers (AWS, GCP, Azure core services), managed services tradeoffs, cost optimization (rightsizing, reserved instances, spot instances), serverless platforms (Lambda, Cloud Run, Vercel), edge platforms (Cloudflare Workers, Fastly Compute).

Key questions: When is Kubernetes overkill? Why does every senior engineer have strong feelings about Terraform vs Pulumi? What does "infrastructure as code" actually mean in practice?

Free resource: KodeKloud's free courses plus AWS Free Tier hands-on labs.

Phase 16: Practice problems (ongoing forever)

What you learn: Tier 1 (start here, 2 to 3 problems per week): URL shortener (Bitly), pastebin, distributed rate limiter, key-value store, web crawler, distributed cache, leaderboard service, distributed ID generator (Snowflake). Tier 2 (after Tier 1 feels easy): Twitter/Instagram feed, WhatsApp/Slack chat, notification system (push + email + SMS), search autocomplete (typeahead), Yelp/location-based service, Reddit-style forum, parking lot system, file upload service, online judge (LeetCode-lite). Tier 3 (interview-ready): YouTube/Netflix video streaming, Uber/Lyft ride sharing, DoorDash/Swiggy delivery, Dropbox/Google Drive file sync, Google Docs collaborative editing, Zoom video conferencing, Spotify music streaming, Airbnb booking, stock exchange trading platform, distributed task scheduler at scale, search engine, ad serving system, recommendation system, payment system (Stripe-like). Tier 4 (specialized, ML-flavored): distributed model training, feature store, model serving infra, A/B testing platform, real-time bidding, Kubeflow-style ML pipeline, vector search at scale.

Key questions: How would you design WhatsApp to handle 2 billion users? What changes if Twitter prioritizes consistency over availability? How does YouTube serve a viral video to a million viewers in 30 seconds?

Free resource: HelloInterview free Answer Keys plus use ChatGPT/Claude/DeepSeek as a free unlimited mock interviewer with the prompt: "Act as a Google staff engineer interviewing me for [problem]. Push back, ask follow-ups, do not give the answer until I have worked through it."

Foundational Books

Designing Data-Intensive Applications by Martin Kleppmann (the bible). **System Design Interview Vol 1 and Vol 2** by Alex Xu. **Building Microservices** by Sam Newman. **Database Internals** by Alex Petrov. **Designing Distributed Systems** by Brendan Burns (free PDF from Microsoft). **Site Reliability Engineering** by Google (free at sre.google). **Release It!** by Michael Nygard. **The DynamoDB Book** by Alex DeBrie.

Foundational Papers (read these once you hit Phase 8)

Google File System, MapReduce, BigTable, Dynamo (Amazon), Spanner, Chubby, ZooKeeper, Raft, Kafka original paper, Borg, Cassandra, F1.

Top YouTube Channels

ByteByteGo, Gaurav Sen, Hussein Nasser, Arpit Bhayani, Hello Interview, Jordan Has No Life, CodeKarle, Tech Dummies (Narendra L), MIT OpenCourseWare 6.824.

Top GitHub Repositories

donnemartin/system-design-primer, ashishps1/awesome-system-design-resources, prasadjayar/low-level-design-primer, karanpratapsingh/system-design.

Free AI Tools That Replace Paid Courses

Mock interviewers: ChatGPT, Claude, Gemini, DeepSeek, Mistral Le Chat (all free tiers, prompt them as senior engineers conducting an interview). **Study companions:** NotebookLM (drop in DDIA chapters or distributed systems papers, get audio overviews and answer questions), Perplexity for research. **Diagram generators:** DiagramGPT by Eraser, Clouday AI System Design Maker, InfraSketch, Excalidraw with AI, Mermaid Chart AI. **Code understanding:** DeepWiki for any open source repo (Kafka, Redis, Cassandra source code explained).

Suggested 6-Month Study Schedule

Month 1: Phases 1 to 4 (foundations and databases). **Month 2:** Phases 5 to 7 (APIs, core components, scaling). **Month 3:** Phases 8 to 10 (distributed systems theory, async processing, architecture patterns). **Month 4:** Phases 11 to 14 (reliability, performance, security, observability). **Month 5:** Phase 15 (DevOps and infrastructure) plus start Tier 1 practice problems. **Month 6:** Tier 2 and Tier 3 practice problems with daily mock interviews using AI tools.

How to Use This Roadmap

Do not skip phases. Each one builds on the previous. Spend the suggested time on each phase but do not move on just because the time elapsed; move on when you can explain the topic to someone else without notes. Build small projects after every two phases (a basic web server after Phase 2, a database-backed API after Phase 5, a sharded key-value store after Phase 7, a distributed counter after Phase 9). The fastest path to mastery is teaching: write a blog post or shoot a Reel explaining each concept after you learn it. If you cannot explain it simply, you have not learned it yet.